

Processamento de Alto Desempenho (PAD)

Relatório de Trabalho 1 – Distância entre Vetores

Implementação, otimização e análise experimental de desempenho

Xavier Paulino Sebastião

Agosto de 2026

Resumo

Este trabalho avalia o cálculo da distância quadrática entre um vetor de referência e um conjunto de vetores reais e, em uma segunda etapa, compara a implementação original com uma versão otimizada. A versão de referência, denominada `baseline_v3`, utiliza um laço escalar, enquanto a versão `optimized_v4` explora SIMD AVX, desenrolamento do laço, quatro acumuladores vetoriais independentes e qualificadores `restrict`, mantendo execução *single-thread* e as mesmas flags principais de compilação. A campanha comparativa considera oito tamanhos de vetor, de $T = 32$ a $T = 4096$, e mede as duas implementações sob as mesmas condições experimentais. A versão otimizada apresentou ganho em todos os tamanhos, com *speedup* entre aproximadamente $1,32\times$ e $1,85\times$. O maior ganho ocorreu em $T = 2048$, enquanto em $T = 4096$ o *speedup* diminuiu para cerca de $1,66\times$. Os resultados mostram que a otimização reduz de forma consistente o custo por elemento e aumenta a taxa de processamento, embora o ganho não permaneça constante em toda a faixa de tamanhos.

1 Introdução

O problema consiste em calcular, para cada vetor x_i de um conjunto X com N vetores, a distância quadrática em relação a um vetor de referência q . Cada vetor contém T valores reais, e a distância é definida por

$$D(q, x_i) = \sum_{j=0}^{T-1} (q_j - x_{ij})^2, \quad i = 0, \dots, N-1. \quad (1)$$

Cada resultado é armazenado após o processamento do respectivo vetor. Mantendo N constante e variando T , a quantidade de elementos processados por execução é $N \times T$ e o trabalho aritmético cresce linearmente com T .

A primeira etapa do trabalho caracterizou o comportamento da implementação escalar para diferentes valores de T . A segunda etapa introduz uma versão otimizada do mesmo cálculo e avalia quanto essa otimização altera o tempo de execução, as taxas de processamento, o custo por elemento e a variabilidade. Em avaliações de desempenho, múltiplas observações e a caracterização da dispersão são importantes para evitar conclusões baseadas em execuções isoladas [2, 3, 8].

2 Implementações avaliadas

2.1 Baseline v3

A implementação de referência foi desenvolvida em C++ [7], com dados em precisão dupla e armazenamento contíguo da matriz X . Para cada vetor, o kernel percorre sequencialmente os T elementos e acumula a soma dos quadrados das diferenças:

```

for (size_t i = 0; i < N; ++i) {
    const double* row = X + i * T;
    double sum = 0.0;
    for (size_t j = 0; j < T; ++j) {
        const double d = q[j] - row[j];
        sum += d * d;
    }
    distances[i] = sum;
}

```

Essa versão é mantida no código da Parte 2 sem alteração funcional e é identificada como **baseline_v3**. A manutenção do kernel original dentro do mesmo executável permite comparar as duas implementações na mesma campanha, evitando utilizar apenas medições obtidas em momentos distintos.

2.2 Optimized v4

A versão otimizada preserva o mesmo layout de dados, o mesmo cálculo matemático e o mesmo orçamento de recursos: a execução continua *single-thread*. A otimização foi realizada no laço interno e combina quatro técnicas. Primeiro, os ponteiros do kernel são declarados com `__restrict__`, reduzindo a ambiguidade de aliasing para o compilador. Segundo, quando AVX está disponível, quatro valores `double` são processados por instrução vetorial. Terceiro, o laço é desenrolado em blocos de 16 elementos. Quarto, são usados quatro acumuladores vetoriais independentes, diminuindo a cadeia serial de dependência da redução.

O trecho principal da versão AVX é representado por:

```

__m256d acc0 = _mm256_setzero_pd();
__m256d acc1 = _mm256_setzero_pd();
__m256d acc2 = _mm256_setzero_pd();
__m256d acc3 = _mm256_setzero_pd();

for (; j + 15 < T; j += 16) {
    // quatro blocos de 4 doubles
    // load q, load x, subtracao, multiplicacao e acumulacao
}

```

Após o bloco principal, vetores AVX completos remanescentes são processados em grupos de quatro elementos e um laço escalar trata o restante. Assim, a implementação não depende de T ser múltiplo de 16. A ordem da soma difere da versão escalar; por isso, a validação de corretude compara as duas implementações com tolerância numérica, em vez de exigir igualdade bit a bit. A compilação mantém C++20, -O3 e -march=native, sem -ffast-math. Otimizações SIMD e transformações de laços são mecanismos diretamente relacionados ao código gerado e ao uso da microarquitetura [4].

3 Metodologia experimental

3.1 Plataforma e ambiente

As aferições foram executadas em um Lenovo ThinkPad T530 equipado com processador Intel Core i5-3320M a 2,60 GHz, arquitetura x86-64, família 6, modelo 58. A CPU disponibiliza AVX, SSE4.1 e SSE4.2. A compilação foi realizada com GCC 10.5.0, usando /usr/bin/g++. O sistema operacional utilizado foi GNU/Linux Ubuntu 24.04.4 LTS. Os scripts de análise e geração de gráficos foram executados no ambiente Conda `t0_vector_distance`.

3.2 Campanha comparativa

A comparação da Parte 2 mede `baseline_v3` e `optimized_v4` novamente na mesma campanha. Para cada bloco experimental, as 16 combinações formadas por duas implementações e oito valores de T são ordenadas de forma pseudoaleatória e determinística. Essa estratégia reduz a associação sistemática entre uma variante e o momento temporal da execução, aspecto relevante porque condições aparentemente secundárias da configuração experimental podem introduzir viés em medições de desempenho [6].

Na campanha representada pelos resultados comparativos, $N = 8192$ permaneceu constante para as duas implementações e para todos os valores de

$$T \in \{32, 64, 128, 256, 512, 1024, 2048, 4096\}.$$

Cada combinação (implementação, T) foi aferida repetidamente, e as estatísticas apresentadas utilizam a mediana como medida central. O uso de repetições permite caracterizar a variabilidade e reduzir a dependência de uma única observação [8].

A região cronometrada contém exclusivamente o cálculo e o armazenamento das N distâncias. Alocação, geração e inicialização dos dados, aquecimento, checksum, escrita de arquivos e apresentação dos resultados permanecem fora do intervalo medido. A temporização utiliza `CLOCK_MONOTONIC_RAW` [1].

3.3 Índices de desempenho

Além do tempo de execução t , foram calculadas as taxas

$$P_v = \frac{N}{t}, \quad P_e = \frac{NT}{t}, \quad (2)$$

onde P_v representa vetores processados por segundo e P_e elementos processados por segundo. O custo temporal por elemento é dado por

$$C_e = \frac{t \times 10^9}{NT} \quad \text{ns/elemento}. \quad (3)$$

Para quantificar diretamente o efeito da otimização, foi calculado

$$S(T) = \frac{t_{\text{baseline}}(T)}{t_{\text{optimized}}(T)}, \quad (4)$$

em que $S(T) > 1$ representa ganho de desempenho. A redução percentual do tempo é

$$R(T) = 100 \left(1 - \frac{t_{\text{optimized}}(T)}{t_{\text{baseline}}(T)} \right). \quad (5)$$

4 Resultados

4.1 Resumo quantitativo

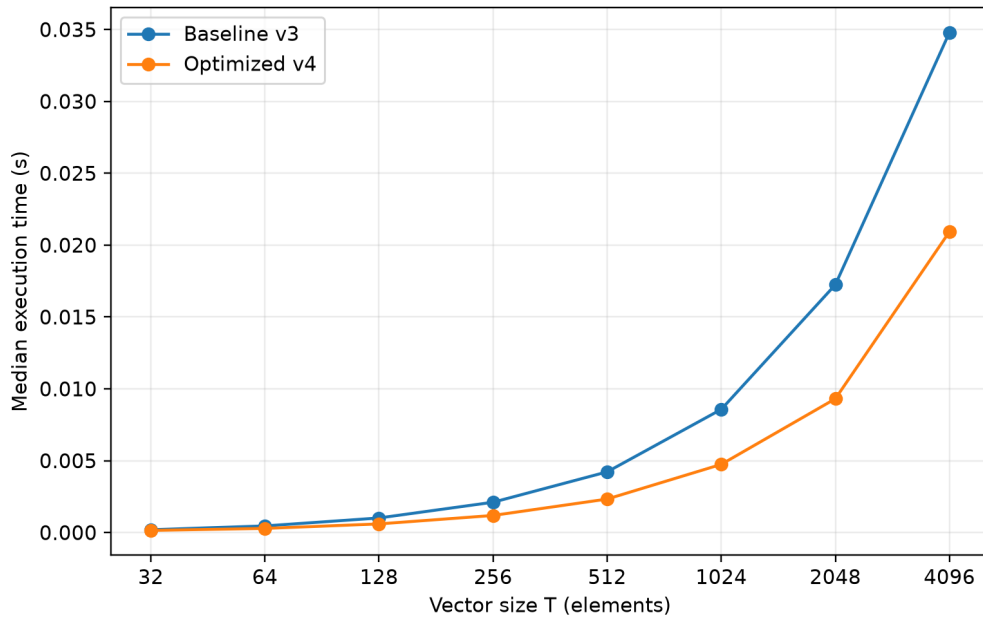
A Tabela 1 reúne as principais métricas das duas implementações. A versão otimizada foi mais rápida em todos os tamanhos avaliados. O ganho não é uniforme: o *speedup* cresce de aproximadamente $1,32\times$ em $T = 32$ até $1,85\times$ em $T = 2048$ e diminui para cerca de $1,66\times$ em $T = 4096$.

Tabela 1: Comparação entre a implementação baseline v3 e a optimized v4.

T	Tempo mediano (ms)		P_e (10^9 elem./s)		ns/elem.		Speedup	Redução (%)
	v3	v4	v3	v4	v3	v4		
32	0,198	0,150	1,325	1,752	0,755	0,571	1,32	24,4
64	0,463	0,290	1,132	1,810	0,883	0,553	1,60	37,4
128	1,015	0,602	1,033	1,742	0,968	0,574	1,69	40,7
256	2,111	1,190	0,993	1,762	1,007	0,568	1,77	43,6
512	4,225	2,333	0,993	1,798	1,007	0,556	1,81	44,8
1024	8,570	4,742	0,979	1,769	1,022	0,565	1,81	44,7
2048	17,247	9,305	0,973	1,803	1,028	0,555	1,85	46,1
4096	34,783	20,917	0,965	1,604	1,037	0,623	1,66	39,9

4.2 Tempo de execução

A Figura 1 mostra que o tempo cresce com T nas duas versões, mas a curva otimizada permanece abaixo da baseline em toda a faixa. Em $T = 32$, o tempo mediano é aproximadamente 0,198 ms na v3 e 0,150 ms na v4. Em $T = 4096$, os valores são aproximadamente 34,783 ms e 20,917 ms, respectivamente.

**Figura 1:** Tempo mediano de execução das versões baseline v3 e optimized v4.

4.3 Taxas de processamento

A taxa de vetores por segundo aumenta com a otimização em todos os tamanhos, como mostra a Figura 2. A diferença é especialmente visível nos tamanhos intermediários, nos quais a v4 conclui aproximadamente entre $1,7\times$ e $1,9\times$ mais vetores por unidade de tempo.

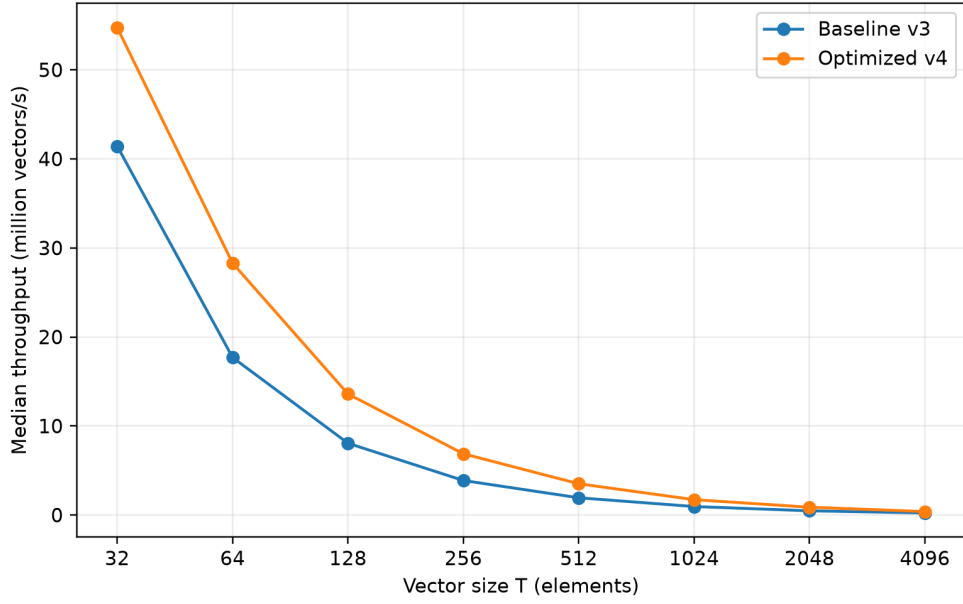


Figura 2: Taxa mediana de processamento de vetores.

A Figura 3 apresenta a taxa de elementos, que é mais adequada para comparar o custo por unidade de trabalho. A baseline varia de aproximadamente 1,33 bilhão de elementos/s em $T = 32$ para 0,97 bilhão em $T = 4096$. A versão otimizada permanece próxima de 1,75–1,81 bilhão de elementos/s entre $T = 32$ e $T = 2048$, caindo para aproximadamente 1,60 bilhão de elementos/s em $T = 4096$.

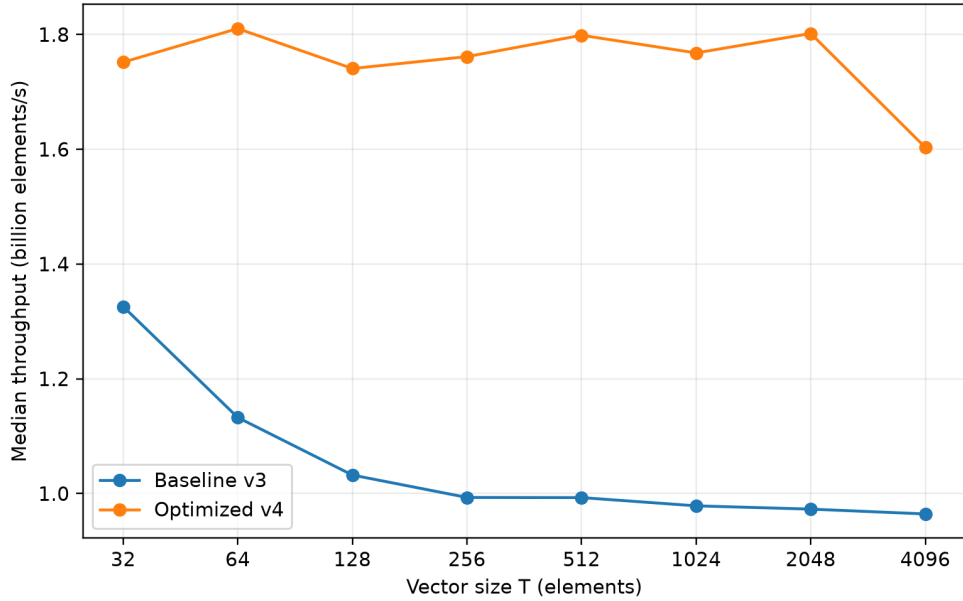


Figura 3: Taxa mediana de processamento de elementos.

4.4 Custo por elemento

O custo por elemento evidencia diretamente o efeito da otimização. Na baseline, esse custo cresce de aproximadamente 0,755 ns/elemento em $T = 32$ para 1,037 ns/elemento em $T = 4096$. Na v4, permanece entre aproximadamente 0,553 e 0,574 ns/elemento de $T = 32$ a $T = 2048$, com aumento para cerca de 0,623 ns/elemento no maior tamanho.

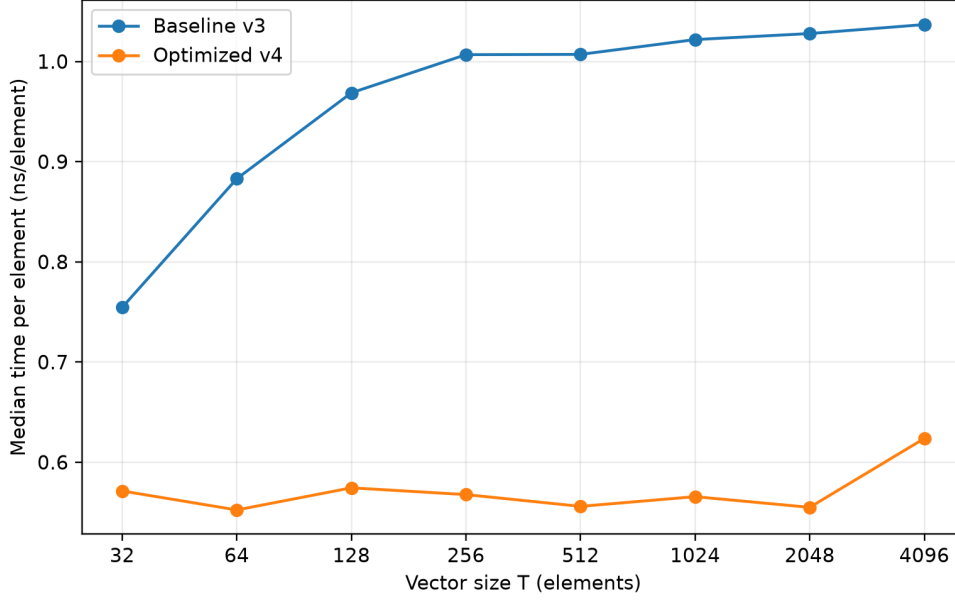


Figura 4: Custo mediano por elemento nas duas implementações.

A estabilidade relativa do custo por elemento na v4 entre $T = 64$ e $T = 2048$ indica que a nova organização do laço reduz parte do custo observado na implementação escalar. Entretanto, a elevação em $T = 4096$ mostra que o ganho não é independente do tamanho da entrada. Como o experimento não coleta contadores de hardware ou largura de banda de memória, não é possível identificar de forma conclusiva se essa redução do ganho decorre de cache, memória principal, prefetch, frequência da CPU ou outro fator microarquitetural. A infraestrutura `perf_event` do Linux permitiria medir eventos como ciclos, instruções e faltas de cache, mas esses indicadores não fazem parte dos dados apresentados [5].

4.5 Speedup

A Figura 5 resume o efeito da otimização. O ganho cresce de $1,32\times$ em $T = 32$ para aproximadamente $1,60\times$ em $T = 64$ e supera $1,77\times$ a partir de $T = 256$. O maior valor observado é aproximadamente $1,85\times$ em $T = 2048$. Em $T = 4096$, o *speedup* cai para aproximadamente $1,66\times$, mas permanece substancialmente acima de 1.

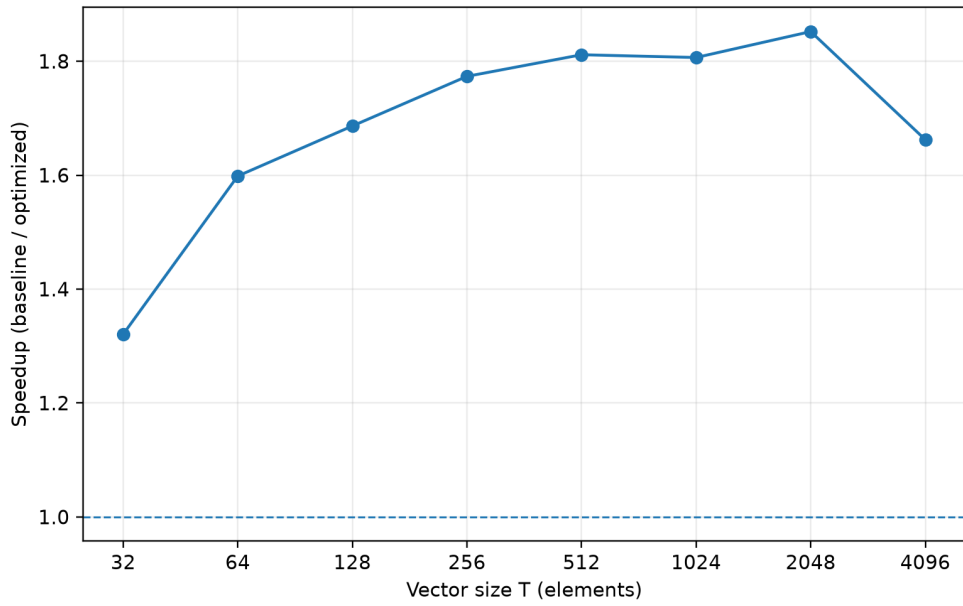


Figura 5: Speedup da versão optimized v4 em relação à baseline v3. A linha tracejada representa ausência de ganho.

4.6 Variabilidade

A Figura 6 apresenta as distribuições das aferições. As duas versões mostram dispersão relativamente pequena na maior parte dos tamanhos, com observações superiores isoladas em alguns casos. Em $T = 4096$, a baseline apresenta outliers acima do núcleo principal da distribuição; a versão otimizada também apresenta observações elevadas, porém em uma escala de tempo menor. As observações foram preservadas, evitando reduzir artificialmente a variabilidade da campanha.

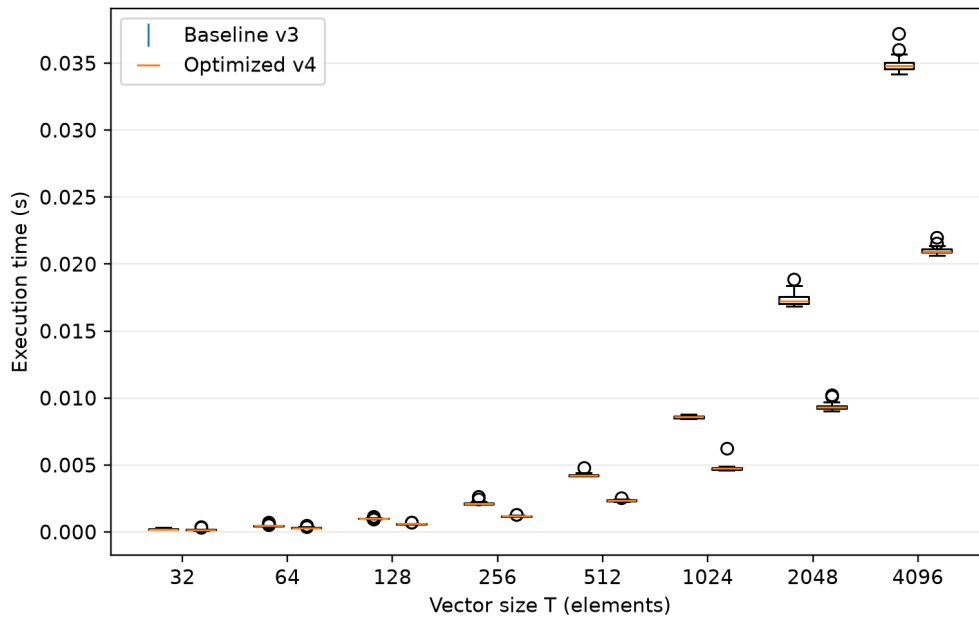


Figura 6: Distribuição dos tempos de execução das duas implementações.

5 Discussão

Os resultados mostram que a otimização produz ganho consistente em toda a faixa de tamanhos, mas não um fator constante. Esse comportamento é compatível com a natureza das mudanças realizadas no kernel. A versão escalar mantém uma única variável de acumulação, de modo que cada atualização de `sum` depende do valor produzido na iteração anterior. A v4 utiliza quatro acumuladores AVX independentes no bloco principal, permitindo maior paralelismo em nível de instrução e processando múltiplos elementos simultaneamente. O desenrolamento também reduz a frequência relativa do controle do laço.

O ganho de aproximadamente $1,8\times$ nos tamanhos intermediários e grandes é inferior ao fator quatro correspondente à largura de um registrador AVX com valores `double`. Isso não representa uma anomalia. O tempo total do kernel não é determinado apenas pela quantidade de operações aritméticas: carregamentos, armazenamento do resultado, redução horizontal, controle de laço, latências de instruções e movimento de dados também participam do custo. Além disso, a implementação já era compilada com `-O3 -march=native`, de modo que parte das otimizações de baixo nível podia ser explorada pelo compilador mesmo antes da introdução explícita do kernel AVX.

A queda de *speedup* em $T = 4096$ merece atenção. Nesse ponto, tanto o throughput da v4 quanto o ganho relativo diminuem, enquanto o custo por elemento aumenta de aproximadamente 0,555 ns em $T = 2048$ para 0,623 ns em $T = 4096$. O resultado demonstra uma mudança mensurável, mas não identifica sua causa. A hierarquia de memória e o prefetch de hardware podem influenciar percursos sequenciais desse tipo [4]; contudo, sem contadores de cache, tráfego de memória ou frequência instantânea, associar a queda de ganho especificamente a um desses mecanismos iria além da evidência coletada.

A comparação também foi estruturada para reduzir um problema metodológico presente em uma comparação simples entre duas campanhas independentes. Baseline e versão otimizada foram medidas novamente no mesmo período experimental e intercaladas em blocos, reduzindo a possibilidade de que uma diferença temporal da máquina seja confundida com o efeito da otimização. Essa prática é coerente com recomendações de benchmarking que enfatizam repetição, controle de variáveis e análise explícita de variabilidade [2, 8, 3].

6 Conclusão

A otimização da implementação produziu melhoria de desempenho em todos os tamanhos avaliados sem recorrer a paralelismo entre cores. A versão *optimized v4* manteve o mesmo cálculo, o mesmo layout de dados e execução *single-thread*, concentrando as mudanças no kernel por meio de AVX, desenrolamento do laço, acumuladores independentes e `restrict`.

O *speedup* variou de aproximadamente $1,32\times$ a $1,85\times$. O maior ganho foi observado em $T = 2048$, com redução de tempo de aproximadamente 46,1%. Em $T = 4096$, o ganho caiu para cerca de $1,66\times$, ainda correspondendo a redução de aproximadamente 39,9% no tempo mediano. A taxa de elementos da versão otimizada permaneceu próxima de 1,8 bilhão de elementos/s na maior parte da faixa e o custo por elemento ficou próximo de 0,56 ns até $T = 2048$.

Os resultados confirmam que a reorganização do kernel reduz de forma substancial o custo computacional da operação, mas também mostram que a eficiência da otimização depende do tamanho da entrada. A queda relativa observada no maior caso não pode ser explicada de forma conclusiva apenas pelos tempos coletados. Assim, a evidência experimental sustenta o ganho da versão otimizada, mas não uma atribuição causal específica para a mudança de comportamento em $T = 4096$.

Referências

- [1] G. Buttazzo and G. Lipari, “Ptask: An educational C library for programming real-time systems on Linux,” in *2013 IEEE 18th Conference on Emerging Technologies & Factory Automation (ETFA)*, pp. 1–8, 2013.
- [2] T. Hoefer and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, 2015.
- [3] C. Curtsinger and E. D. Berger, “Stabilizer: Statistically sound performance evaluation,” *ACM SIGARCH Computer Architecture News*, vol. 41, no. 1, pp. 219–228, 2013.
- [4] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Version 050, 2026. Disponível em: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel64-and-ia32-architectures-optimization.html>.
- [5] M. Kerrisk and Linux man-pages contributors, “perf_event_open(2) – set up performance monitoring,” *Linux man-pages*, 2026. Disponível em: https://man7.org/linux/man-pages/man2/perf_event_open.2.html.
- [6] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!,” *ACM SIGPLAN Notices*, vol. 44, no. 3, pp. 265–276, 2009.
- [7] H. Schildt, *C++: The Complete Reference*. McGraw-Hill, 1998.
- [8] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pp. 63–74, 2013.